



Week1_예습과제_고은비

1장 : 파이썬 기반의 머신러닝과 생태계 이해

- 1 머신러닝의 개념
- 2 파이썬 머신러닝 생태계를 구성하는 주요 패키지
- 3 넘파이
 - 넘파이 ndarray 개요
 - 넘파이 ndarray의 데이터 세트 선택하기
 - 행렬의 정렬
 - 선형대수 연산
- 4 데이터 핸들링 - 판다스
 - DataFrame으로 로딩, 기본 API
 - DataFrame 상호 변환
 - DataFrame의 칼럼 데이터 세트 생성/수정
 - DataFrame데이터 삭제
 - Index 객체
 - 데이터 셀렉션 및 필터링
 - 정렬, Aggregation 함수, GroupBy 적용
 - 결손 데이터 처리
 - apply lambda식 데이터 가공
- 5 정리

1장 : 파이썬 기반의 머신러닝과 생태계 이해

1 머신러닝의 개념



머신러닝

: 애플리케이션을 수정하지 않고도

데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법

⇒ 소프트웨어 코드로 로직을 구성하여 일정한 패턴을 찾기 어려운 경우, 훌륭한 솔루션을 제공

⇒ 데이터 분석 영역의 재편 - 머신러닝 기반의 예측 분석 (Predictive Analysis)

머신러닝의 분류

- 지도학습 (Supervised Learning)
 - 분류 (Classification)
 - 회귀 (Regression)
 - 추천 시스템
 - 시각/음성 감지/인지
 - 텍스트 분석, NLP
- 비지도학습 (Un-supervised Learning)
 - 클러스터링
 - 차원 축소

- 강화학습

데이터 전쟁

- 머신러닝 세상에서는 데이터의 중요성이 무엇보다 커짐
 - 머신러닝의 가장 큰 단점 : 큰 데이터 의존성 (Garbage In, Garbage Out)
 - 다양하고 광대한 데이터를 기반으로 만들어진 머신러닝 모델은 더 좋은 품질을 약속
- ⇒ 데이터를 이해하고 효율적으로 가공, 처리, 추출해 최적의 데이터를 기반으로 알고리즘을 구동할 수 있도록 준비하는 능력이 더 중요.

파이썬 vs R 기반 머신러닝

머신러닝 프로그램을 작성할 수 있는 대표적 오픈 소스 프로그램 언어

- **R**
: 통계 전용 프로그램 언어
 - 개발 언어에 미숙, 통계 분석에 익숙한 현업 사용자라면 추천
 - 오랜 기간 많은 사용자들이 생성하고 검증해온 통계 패키지 보유
- **Python**
: 다양한 영역에서 사용되는 개발 전문 프로그램 언어

➡ 머신러닝을 시작하려는 사람에게는 **Python 추천**

- 전세계 개발자들이 선호
- 풍부한 라이브러리
- Interpreter Language 특성 - 느린 속도 / 뛰어난 확장성, 유연성, 호환성
- 머신러닝 앱과 결합한 다양한 개발 가능 / 다양한 기업 환경으로의 확산 가능
- 딥러닝 프레임워크 (TensorFlow, Keras, PyTorch 등) 파이썬 우선 지원 중

2 파이썬 머신러닝 생태계를 구성하는 주요 패키지

- ✓ 파이썬 언어를 이용해 머신러닝 애플리케이션을 작성하기 위해서는 먼저 관련 패키지들에 친숙해져야한다!

✓ 머신러닝 패키지

- 사이킷런 (Scikit-Learn) : 가장 대표적 머신러닝 패키지

✓ 행렬/선형대수/통계 패키지

: 머신러닝의 이론적 배경은 선형대수 + 통계

- 넘파이(Numpy) : 많은 머신러닝 패키지의 기반이 됨
- 사이파이(SciPy) : 자연과학, 통계를 위한 패키지

✓ 데이터 핸들링

- 판다스(Pandas) : 대표적인 데이터 처리 패키지 / 행렬 기반 데이터(특히 2차원) 처리에 특화 +맷플롯립(Matplotlib)을 호출해 쉽게 시각화 가능

✓ 시각화

- 맷플롯립(Matplotlib) : 파이썬의 대표적인 시각화 패키지
세분화된 API로 효율이 떨어지고 불편할 수 있음

- 시본(Seaborn) : 맷플롯립 기반 → 판다스와의 쉬운 연동, 함축적 API, 다양한 그래프/차트 제공
⇒ 여전히 세밀한 제어는 맷플롯립 API 사용

✓ 아이파이썬 (IPython, Interactive Python) 툴

: 대화형 파이썬 툴, 코드 영역별 개별 수행을 지원

- 주피터 노트북 (Jupyter Notebook)

3 넘파이

✓ Numpy ⇒ Numerical Python

```
import numpy as np
# np로 alias해 임포트하는 것이 관례
```

파이썬에서 선형대수 기반의 프로그램을 만들 수 있도록 지원하는 대표적 패키지

- 루프 없이 대량 데이터의 빠른 배열 연산 속도 보장
- 저수준 언어(C/C++) 기반의 호환 API 제공
- 2차원 형태의 데이터에 대한 부분에서는 판다스의 편리성에 미치지 못함
(넘파이에 대한 이해는 판다스를 이해하는 데 도움이 됨)
- 많은 머신러닝 알고리즘들이 넘파이를 기반으로 작성

➡ 넘파이를 이해하는 것은 파이썬 기반의 머신러닝에서 매우 중요!

📄 넘파이 ndarray 개요

- 넘파이 기반의 데이터 타입은 ndarray
→ 넘파이에서 다차원(Multi-dimension) 배열을 쉽게 생성 + 다양한 연산 수행
- ndarray내 데이터 값은 숫자, 문자열, 불 값 모두 가능
한 개의 객체엔 같은 데이터 타입만 가능
- 리스트 → ndarray 변환시 가장 크기가 큰 데이터 타입으로 형 변환 일괄 적용

array()	파이썬의 리스트와 같은 다양한 인자를 입력받아 ndarray로 변환 → .shape 변수로 행/열 수를 튜플 형태로 확인 → .dtype ndarray 내 데이터 타입 확인
astype()	ndarray내 데이터 값 타입 변경 (메모리 절약 시 이용)
arange()	파이썬의 range와 유사한 기능 - 0~(함수인자값-1) 값을 순차적으로 ndarray의 데이터 값으로 변환
zeros()	함수 인자로 튜플형태의 shape 값을 입력하면 모든 값을 0으로 채운 해당 shape의 ndarray 반환 디폴트 데이터 형식 - float64 → ones() 는 1로 채움
reshape()	ndarray를 특정 차원 및 크기로 변환 (지정된 사이즈로 변경 불가 시 오류 발생) → -1 인자를 사용해 원래 ndarray와 호환되는 새로운 shape로 변환

📄 넘파이 ndarray의 데이터 세트 선택하기

1 특정한 데이터만 추출	[]	원하는 위치의 인덱스 값을 지정하면 해당 위치의 데이터 반환 🔍 다차원 ndarray에서는 콤마로 인덱스를 분리
----------------------	-----------	---

		ex. (2,1) = (row=2, col=1) = (axis0=2, axis1=1) axis0이 로우 방향 축, axis1이 칼럼 방향 축
2 슬라이싱(Slicing)	:	연속된 인덱스 상의 ndarray를 추출하는 방식 → 기호 사이에 시작 인덱스와 종료 인덱스 표시 → 시작 ~ 종료-1 위치의 데이터를 반환 1. 기호 앞에 시작 인덱스를 생략하면 자동으로 0으로 간주 2. 기호 뒤에 종료 인덱스를 생략하면 맨 마지막 인덱스로 간주 3. 기호 앞/뒤에 시작/종료 인덱스를 생략하면 자동으로 맨 처음/마지막 인덱스로 간주
3 팬시 인덱싱 (Fancy Indexing)		일정한 인덱스 집합을 리스트 또는 ndarray 형태로 지정해 해당 위치에 있는 데이터의 ndarray를 반환 ex. array[[0,1],0:2] → 0,1 로우 + 0,1 컬럼
4 불린 인덱싱 (Boolean Indexing)		특정 조건에 해당하는지 True/False 값 인덱싱 집합을 기반으로 True에 해당하는 인덱스 위치의 데이터의 ndarray를 반환 🔍 인덱스를 지정하는 [] 내에 조건문을 그대로 기재 (ndarray 객체에 조건식만 붙이면 False, True로 이루어진 ndarray 객체가 반환됨) step1 ndarray 필터링 조건을 대괄호 안에 기재 step2 False값은 무시, True 값에 해당하는 인덱스 값만 저장 step3 저장된 인덱스 데이터 세트로 ndarray 조회

📄 행렬의 정렬

np.sort()	넘파이에서 sort()를 호출하는 방식 원 행렬은 그대로 유지 / 정렬된 행렬을 반환 🔍 올림차순이 디폴트 → 내림차순 정렬 시 np.sort()[::-1] 🔍 2차원 이상일 경우 axis 축 값 설정을 통해 방향 설정
ndarray.sort()	행렬 자체에서 sort()를 호출하는 방식 행렬 자체를 정렬한 형태로 변환 / 반환값은 None
argsort()	원본 행렬이 정렬 되었을 때, 기존 원본 행렬의 원소에 대한 인덱스 필요 시 사용 🔍 내림차순 정렬 시 np.argsort()[::-1]

넘파이의 ndarray는 판다스의 DataFrame 칼럼과 같은 메타 데이터를 가질 수 없어 실제 값과 그 값이 뜻하는 메타 데이터를 별도의 ndarray로 각각 가져야함 함.

이때 연관된 두 ndarray에서의 정렬된 출력을 원할 때 argsort()가 효과적

📄 선형대수 연산

np.dot()	행렬 내적 (행렬 곱) np.dot(A, B) → 행렬 A,B의 행렬곱 반환
transpose()	전치 행렬 np.transpose(A) → 행렬 A의 전치행렬 반환

4 데이터 핸들링 - 판다스



판다스

```
import pandas as pd
# pd로 alias해 임포트하는 것이 관례
```

데이터 처리를 위한 가장 인기 있는 라이브러리

- 2차원 데이터를 효율적으로 가고/처리할 수 있는 다양한 기능을 제공
 - 리스트, 컬렉션, 넘파이, CSV 등의 파일을 쉽게 DataFrame으로 변경
 - DataFrame : 여러 개의 행, 열로 이루어진 2차원 데이터를 담는 구조체
 - Index : RDBMS의 PK처럼 개별 데이터에 대한 고유 식별 key값
 - Series : DataFrame 처럼 Index를 키값으로 갖지만, 칼럼이 하나 뿐
- 여러개의 Series로 이루어진 것이 DataFrame



DataFrame으로 로딩, 기본 API

판다스는 다양한 포맷으로 된 파일을 DataFrame으로 로딩 가능

<code>read_csv()</code>	CSV 뿐만 아니라 어떠한 필드 구분 문자 기반의 파일 포맷도 변환 (기본은 콤마) ex. <code>read_csv('filepath_of_buffer', sep='\t')</code> 🔍 별다른 파라미터 지정이 없으면, 파일의 첫 로우를 칼럼명으로 인지, 변환
<code>read_table()</code>	기본 필드 구분 문자가 탭 ('\t')
<code>read_fwf()</code>	Fixed Width, 고정 길이 기반의 칼럼 포맷 로딩
<code>.head(n)</code>	맨 앞 n개의 로우 반환 (디폴트는 5)
<code>.shape</code>	DataFrame의 행과 열의 수를 튜플 형태로 반환
<code>.info()</code>	DataFrame의 메타데이터 조회 (칼럼 타입, Null 데이터 개수, 데이터 분포도 등)
<code>.describe()</code>	칼럼별 숫자형 데이터 값의 n-percentile 분포도 / 평균값 / 최댓값 / 최솟값 🔍 오직 숫자형 데이터(int, float ...) 칼럼의 분포도만 조사하며 object 타입은 제외 🔍 데이터 분포도는 머신러닝 알고리즘 성능 향상에 있어 중요한 요소 🔍 칼럼이 숫자형 카테고리 칼럼 (범주형 칼럼)인지 판단 가능

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

- count : Not Null인 데이터 건수
- mean / std : 평균 / 표준편차
- min / max : 최솟값 / 최댓값
- 25% / 50% / 75% : percentile 값



Survived의 경우 min 0, 25~75% 0, max 1이므로 0과 1으로 이루어진 숫자형 카테고리 칼럼이구나!
(Pclass 도 마찬가지)

<code>.value_counts()</code>	DataFrame['col_name'].value_counts() → 해당 칼럼 데이터 세트가 Series 형태로 반환. 해당 칼럼값의 유형과 건수 확인 🔍 칼럼 값별 데이터 건수를 반환하므로 고유 칼럼 값을 식별자(index)로 사용 🔍 문자열도 인덱스로 사용 가능 (단, 고유성이 보장되어야 함) 🔍 Null값을 무시하고 결과값을 내놓음 (dropna = False로 설정하면 포함)
------------------------------	--

DataFrame 상호 변환

사이킷런의 많은 API는 기본적으로 넘파이의 ndarray를 입력 인자로 사용하는 경우가 대부분

➡ 넘파이 ndarray, 리스트, 딕셔너리를 DataFrame으로 상호 변환해보자

리스트, 넘파이 ndarray, 딕셔너리 ➡ DataFrame

- DataFrame은 리스트, 넘파이 ndarray와 다르게 칼럼명을 가지고 있음
 - 편하게 데이터 핸들링 가능
 - DataFrame으로 변환 시 칼럼명 지정 필요 (or 자동 할당)

data	리스트, 딕셔너리, 넘파이 ndarray를 입력 받는 생성 인자
columns	칼럼명 리스트를 입력 받는 생성 인자 <code>pd.DataFrame(list, cols=col_name)</code>

- 딕셔너리의 경우, Key는 칼럼명으로, Value는 키에 해당하는 칼럼 데이터로 변환
 - 키는 문자열, 값은 리스트(또는 ndarray) 형태로 구성

DataFrame ➡ 리스트, 넘파이 ndarray, 딕셔너리

ndarray로 변환	DataFrame 객체의 values 이용	<code>array=df.values</code>
리스트로 변환	values로 얻은 ndarray에 tolist() 호출	<code>list=df.values.tolist()</code>
딕셔너리로 변환	DataFrame 객체의 to_dict() 이용 인자로 'list' 입력 시 값이 리스트형으로 반환	<code>dict = df.to_dict('list')</code>

DataFrame의 칼럼 데이터 세트 생성/수정

- 칼럼 추가
 - DataFrame `[]` 내에 새로운 칼럼명 입력 후 값 할당
- 기존 칼럼 활용
 - 새로운 칼럼 Series를 만들 때, 기존 칼럼의 Series 데이터 활용 가능

`ex. df['new'] = df['orig']*10`

- 기존 칼럼 값 일괄 업데이트
 - 원하는 칼럼명을 입력 후 값 할당

DataFrame데이터 삭제

`drop()` 메서드 이용

`DataFrame.drop(labels=None, axis=0, columns=None, level=None, inplace=False, errors='raise')`

labels	■ 로우 삭제 시 인덱스 입력 <code>[0,1,2]</code> ■ 칼럼 삭제 시 칼럼명 입력 <code>['age','col2']</code>
axis	■ 0 입력 : 로우 축 방향으로 드롭 = 특정 <u>로우</u> 를 드랍 (default) ■ 1 입력 : 칼럼 축 방향으로 드롭 = 특정 <u>칼럼</u> 을 드랍
inplace	■ False : 자신의 DataFrame의 데이터는 유지, 삭제된 결과만 반환 (default) ■ True : 자신의 DataFrame 데이터도 삭제 → 반환값은 None

Index 객체

: DataFrame, Series의 레코드를 고유하게 식별하는 객체

⇒ `DataFrame.index` 또는 `Series.index` 속성을 통해 추출 가능

- 식별성 데이터를 1차원 array로 가지고 있음
- ndarray와 유사하게 단일 값 반환 및 슬라이싱 가능
- 단, 객체 변경은 불가
- Series 객체에 연산 함수 적용 시, Index는 연산에서 제외 - 오직 식별용
- `reset_index()` 메서드 수행시 새로운 인덱스 할당 + 기존 인덱스는 'index' 칼럼으로 추가
 - Series에 적용 시 DataFrame 반환
 - `drop = True` 로 설정 시 기존 인덱스 삭제

데이터 선택 및 필터링

판다스의 데이터 선택과 필터링은 넘파이와 유사한 부분도 있고, 다른 부분도 있기에 주의가 필요

개별 또는 여러개의 칼럼 값 전체를 추출하고자 한다면 `df['칼럼명']` 만으로 충분

<code>[]</code>	■ DataFrame 바로 뒤의 <code>[]</code> 연산자는 넘파이와 Series의 연산자와 다름 ■ 괄호 내 입력값은 칼럼명(또는 리스트)을 지정해 칼럼 지정에 사용하거나 불린 인덱스 용도로만 사용해야함 ■ <code>DataFrame[0:2]</code> 와 같이 슬라이싱 연산으로 추출하는 방식은 권장하지 않음
<code>iloc[]</code>	위치(Location) 기반 인덱싱 <code>iloc[행 위치 정수값, 열 위치 정수값]</code> ■ 좌표를 정수값 형태로 입력 ■ 슬라이싱 / 팬시 인덱싱 제공하지만, 불린 인덱싱은 불가
<code>loc[]</code>	명칭(Label)기반 인덱싱 <code>iloc[인덱스값, 칼럼명]</code> ■ 슬라이싱 기호 적용시 <u>종료값 포함</u>

`iloc[]` 연산 유형

<code>df.iloc[1,0]</code>	두번째 행의 첫번째 열 위치에 있는 단일 값 반환
<code>df.iloc[0:2, [0,1]]</code>	0:2 슬라이싱 범위의 첫번째에서 두번째 행과 1번째, 2번째 열에 해당하는 DataFrame 반환
<code>df.iloc[0:2, 0:3]</code>	0:2 슬라이싱 범위의 첫번째에서 두번째 행의 0:3 슬라이싱 범위의 첫번째 부터 세번째열 범위에 해당하는 DataFrame 반환
<code>df.iloc[:]</code> <code>df.iloc[:,:]</code>	전체 DataFrame 반환
<code>df.iloc[:,-1]</code>	맨 마지막 칼럼 데이터 반환
<code>df.iloc[:, :-1]</code>	맨 마지막 칼럼을 제외한 모든 데이터 반환

`loc[]` 연산 유형

<code>df.loc['one','Name']</code>	인덱스 값 one인 행의 Name 칼럼의 단일값 반환
<code>df.loc['one':'two', ['Name','Year']]</code>	인덱스 값 one 부터 two까지 행의 Name과 Year칼럼에 해당하는 DataFrame 반환
<code>df.loc['one':'two', ['Name':'Gender']]</code>	인덱스 값 one 부터 two까지 행의 Name부터 Gender칼럼까지의 DataFrame 반환

<code>df.loc[:]</code>	전체 DataFrame 반환
<code>df.loc[data_df.Year >= 2014]</code>	iloc과 다르게 불린 인덱싱 가능 Year 칼럼이 2014 이상인 모든 데이터 추출

불린 인덱싱

- 불린 인덱싱의 반환 값은 DataFrame → 원하는 칼럼명만 별도 추출 가능
- 여러개의 복합 조건 결합 적용 가능
 - and : &
 - or : |
 - Not : ~
- 개별 조건 변수 할당 뒤, 변수들을 불린 인덱싱에 활용 가능

정렬, Aggregation 함수, GroupBy 적용

<code>sort_values()</code>	DataFrame과 Series의 정렬을 위해 사용 ■ <code>by</code> → 정렬 수행 기준 칼럼 ■ <code>ascending</code> → False로 설정하면 내림차순 (True가 디폴트) ■ <code>inplace</code> → False로 설정하면 원본 그대로 유지, 정렬된 결과만 반환 (False가 디폴트)
Aggregation 함수 <code>count()</code> <code>mean()</code> <code>max()</code> ...	■ DataFrame에서 바로 호출 시 모든 칼럼에 적용 ■ 대상 칼럼만 추출해 적용하면 됨
<code>groupby()</code>	■ <code>by</code> → groupby 기준 칼럼 ■ 특정 칼럼에 aggregation 적용하는 경우 ⇒ <code>groupby()</code>로 반환된 객체에 해당 칼럼 필터링 후 적용 가능 ■ 서로 다른 aggregation 함수를 적용할 경우 ⇒ 반환 객체에 <code>.agg()</code> 내 인자로 입력해 사용 가능 ex. <code>.agg([max,min])</code> ■ <code>agg()</code> 내 입력값으로 딕셔너리 형태로 함수가 적용된 칼럼과 함수를 입력해 복잡한 처리 가능

결손 데이터 처리

판다스는 결손 데이터를 처리하는 편리한 API 제공

- 결손 데이터 : 칼럼에 값이 없는, NULL인 경우 → 넘파이의 NaN으로 표시
- 머신러닝 알고리즘은 NaN값을 처리하지 않음
- NaN값은 함수 연산 시 제외됨

<code>isna()</code>	데이터가 NaN인지 아닌지 출력 (Ture(=1) or False(=0)) <code>sum()</code> 함수를 추가해 결손 데이터 개수 구하기 가능
<code>fillna()</code>	결손 데이터를 인자값으로 대체 <code>ex. df['Cabin'] = df['Cabin'].fillna('C000')</code> ⇒ 반환값을 다시 받거나, <code>inplace=True</code> 파라미터를 추가해야 실제 데이터 세트가 변경됨

apply lambda식 데이터 가공

판다스는 apply함수에 lambda식을 결합해 DataFrame이나 Series의 레코드별로 데이터를 가공하는 기능을 제공

lambda식

: 함수의 선언과 처리를 한 줄의 식으로 변환하는 식

```
lambda_square = lambda x : x ** 2
```

- `:` 로 입력인자와 반환될 계산식을 분리
- 여러 개의 값을 입력 인자로 사용해야 할 경우 `map()` 함수 결합

```
a=[1,2,3]
squares = map(lambda x : x**2, a)
```

➡ DataFrame의 `apply()` 함수의 인자로 람다식 적용해 데이터 가공 가능

```
titanic_df['Child_Adult'] = titanic_df['Age'].apply(lambda x : 'Child' if x <=15 else 'Adult')
```

- `:` 의 오른쪽에 반환값이 있어야하므로 if 식보다 반환 값을 먼저 기술
- else if 문은 지원하지 x
- else if가 필요한 경우나, switch case 문의 경우 별도의 함수를 만들어 반환값으로 지정

5 정리

실제로 ML 모델을 생성하고 예측을 수행하는 데 있어 ML 알고리즘이 차지하는 비중보다 데이터를 전처리하고 적절한 피처를 가공/추출하는 부분이 훨씬 많은 비중을 차지함.

→ 사이킷런 뿐만 아니라 넘파이, 판다스, 맷플롯립/시본과 같은 다양한 패키지에 대한 이해 역시 매우 중요하다!